

Experiment 09 – Stored Procedures

Sahil Goyal | UID: 24BDA70148

Aim

To study stored procedures in PostgreSQL, and to implement both a parameterised procedure that returns a value through an OUT parameter and a procedure that carries out a multi-step business transaction.

Objective

- Write a parameterised procedure that takes an IN parameter and returns a result through an OUT parameter.
- Use RAISE NOTICE to print status messages from within a procedure.
- Write a procedure that validates a condition, then performs inserts and updates as a single transaction.
- Handle both the success path and the failure path within one procedure using IF / ELSE.

Part I: Count Employees by Gender

Given a table of employees, write a procedure that takes a gender as an IN parameter and returns, through an OUT parameter, the number of employees of that gender.

Schema and sample data

```
DROP TABLE IF EXISTS employee_info;

CREATE TABLE employee_info (
    id      SERIAL PRIMARY KEY,
    name    VARCHAR(50) NOT NULL,
    gender  VARCHAR(10) NOT NULL,
    salary  NUMERIC(10,2) NOT NULL,
    city    VARCHAR(50) NOT NULL
);

INSERT INTO employee_info (name, gender, salary, city) VALUES
('Alok', 'Male', 50000.00, 'Delhi'),
('Priya', 'Male', 60000.00, 'Mumbai'),
('Rajesh', 'Female', 45000.00, 'Bangalore'),
('Sneha', 'Male', 55000.00, 'Chennai'),
('Anil', 'Male', 52000.00, 'Hyderabad'),
('Sunita', 'Female', 48000.00, 'Kolkata'),
('Vijay', 'Male', 47000.00, 'Pune'),
('Ritu', 'Male', 62000.00, 'Ahmedabad'),
('Amit', 'Female', 51000.00, 'Jaipur');
```

Procedure

```
CREATE OR REPLACE PROCEDURE sp_get_employees_by_gender(
    IN  p_gender VARCHAR(50),
    OUT p_employee_count INT
)
LANGUAGE plpgsql
AS $$
BEGIN
    SELECT COUNT(id)
    INTO p_employee_count
    FROM employee_info
    WHERE gender = p_gender;

    RAISE NOTICE 'Total employees with gender %: %', p_gender, p_employee_count;
END;
```

```
$$;
```

```
CALL sp_get_employees_by_gender('Male', NULL);
```

Output

Unsaved e

```
1 CREATE OR REPLACE PROCEDURE sp_get_employees_by_gender(  
2     IN p_gender VARCHAR(50),  
3     OUT p_employee_count INT  
4 )  
5 LANGUAGE plpgsql  
6 AS $$  
7 BEGIN  
8     SELECT COUNT(id)  
9     INTO p_employee_count  
10    FROM employee_info  
11    WHERE gender = p_gender;  
12  
13    RAISE NOTICE 'Total employees with gender %: %', p_gender, p_employee_count;  
14 END;  
15 $$;  
16  
17 CALL sp_get_employees_by_gender('Male', NULL);  
18
```

Results Chart

p_employee_count

6

Part II: Process a Product Order

An e-commerce backend needs to process a customer order given a product name and a desired quantity. The procedure should check whether enough stock is available; if so, it should log the sale, calculate the total price, reduce the remaining stock, and increase the quantity sold. If stock is insufficient, it should leave the data untouched and raise a notice instead.

Schema and sample data

The worksheet's procedure refers to products and sales tables that were never defined — added here so the procedure actually runs.

```
DROP TABLE IF EXISTS sales CASCADE;  
DROP TABLE IF EXISTS products CASCADE;  
  
CREATE TABLE products (  
    product_code    VARCHAR(20) PRIMARY KEY,  
    product_name    VARCHAR(100) NOT NULL,  
    price           NUMERIC(10,2) NOT NULL,
```

```

        quantity_remaining INT NOT NULL,
        quantity_sold      INT NOT NULL DEFAULT 0
    );

CREATE TABLE sales (
    sale_id          SERIAL PRIMARY KEY,
    order_date       DATE NOT NULL,
    product_code     VARCHAR(20) REFERENCES products(product_code),
    quantity_ordered INT NOT NULL,
    sale_price       NUMERIC(10,2) NOT NULL
);

INSERT INTO products
(product_code, product_name, price, quantity_remaining, quantity_sold) VALUES
('MB14', 'MacBook Pro 14"', 150000.00, 5, 0),
('IP15', 'iPhone 15', 70000.00, 2, 0);

```

Procedure

```

CREATE OR REPLACE PROCEDURE pr_buy_products(
    IN p_product_name VARCHAR,
    IN p_quantity      INT
)
LANGUAGE plpgsql
AS $$
DECLARE
    v_product_code VARCHAR(20);
    v_price         FLOAT;
    v_count         INT;
BEGIN
    -- Step 1: check if the product exists with enough stock
    SELECT COUNT(*)
    INTO v_count
    FROM products
    WHERE product_name = p_product_name
        AND quantity_remaining >= p_quantity;

    -- Step 2: if sufficient stock, process the order
    IF v_count > 0 THEN
        SELECT product_code, price
        INTO v_product_code, v_price
        FROM products
        WHERE product_name = p_product_name;

        INSERT INTO sales (order_date, product_code, quantity_ordered, sale_price)
        VALUES (CURRENT_DATE, v_product_code, p_quantity, v_price * p_quantity);

        UPDATE products
        SET quantity_remaining = quantity_remaining - p_quantity,
            quantity_sold      = quantity_sold + p_quantity
        WHERE product_code = v_product_code;

        RAISE NOTICE 'PRODUCT SOLD..! Order placed successfully for % unit(s) of %.',
            p_quantity, p_product_name;
    ELSE
        -- Step 3: stock is insufficient
        RAISE NOTICE 'INSUFFICIENT QUANTITY..! Order cannot be processed for % unit(s)
of %.',
            p_quantity, p_product_name;
    END IF;

```

```

END;
$$;

-- success case
CALL pr_buy_products('MacBook Pro 14"', 1);

-- failure case – more units than are in stock
CALL pr_buy_products('iPhone 15', 10);

```

Output

Unsaved e

```

11 BEGIN
15     FROM products
16     WHERE product_name = p_product_name
17         AND quantity_remaining >= p_quantity;
18
19     -- Step 2: if sufficient stock, process the order
20     IF v_count > 0 THEN
21         SELECT product_code, price
22         INTO v_product_code, v_price
23         FROM products
24         WHERE product_name = p_product_name;
25
26         INSERT INTO sales (order_date, product_code, quantity_ordered, sale_price)
27         VALUES (CURRENT_DATE, v_product_code, p_quantity, v_price * p_quantity);
28
29         UPDATE products
30         SET quantity_remaining = quantity_remaining - p_quantity,
31           quantity_sold       = quantity_sold + p_quantity
32         WHERE product_code = v_product_code;

```

Results Chart

Success. No rows returned

Learning Outcome

1. Understood and implemented stored procedures in PostgreSQL.
2. Used a parameterised procedure with IN and OUT parameters to apply business logic.
3. Combined conditional checks, inserts, and updates inside a single procedure to model a real transaction.